📘 Algorithm Deconstruction Journal – Task Priority Sorting Algorithm

🧠 Overview of the Algorithm

The Task Priority Sorting Algorithm is designed to rank tasks based on their importance using a scoring system. Instead of sorting tasks using a single attribute (like priority only), it combines multiple factors such as priority level, due date, completion status, tags, and recent activity. Each task is assigned a numerical score, and tasks are then sorted based on this score in descending order.

🔍 Step-by-Step Breakdown

1. Score Calculation (calculate_task_score)

The algorithm starts by assigning a **base score** depending on task priority:

- LOW → 10
- MEDIUM → 20
- HIGH → 40
- URGENT → 60

This ensures that higher priority tasks start with a higher baseline.

Next, it adjusts the score based on **due date proximity**:

- Overdue → +35
- Due today → +20
- Due in ≤2 days → +15
- Due in ≤7 days → +10

This increases urgency for tasks that are closer to their deadlines.

The algorithm then applies **penalties**:

- DONE tasks → −50
- REVIEW tasks → −15

This ensures completed or near-complete tasks are deprioritized.

It also includes **boosts**:

- Tags like “blocker”, “critical”, “urgent” → +8
- Recently updated (within 1 day) → +5

Finally, the function returns a total score representing task importance.

---

2. Sorting Tasks (sort_tasks_by_importance)

- The algorithm calculates scores for all tasks
- It pairs each task with its score
- It sorts tasks in descending order (highest score first)

This ensures the most important tasks appear at the top.

---

3. Getting Top Tasks (get_top_priority_tasks)

- Calls the sorting function
- Returns only the top N tasks (default = 5)

📊 Visual Representation (Conceptual Flow)

Task → Calculate Score → Add Factors → Final Score

↓

All Tasks → Sort by Score (Descending)

↓

Top N Tasks Selected

💡 Key Insights & Learning Points

- The algorithm uses a **weighted scoring system**, which is more flexible than simple sorting
- It combines multiple real-world factors (urgency, importance, activity)
- The design allows easy extension (e.g., adding new scoring rules)
- The use of both penalties and boosts creates balanced prioritization

---

🔄 Reflection

How AI Explanation Helped

The AI explanation made it easier to understand how multiple factors combine into a single score. Initially, the algorithm looked complex, but breaking it into steps clarified the logic behind each condition.

Difficult Aspects

- Understanding how different weights impact final ranking
- Visualizing how multiple conditions interact together

How I Would Explain It to Another Junior Developer

I would explain it like this:

"Each task gets a score based on how important and urgent it is. High priority tasks and tasks due soon get higher scores, while completed tasks lose points. After calculating scores for all tasks, we simply sort them from highest to lowest to decide what to work on first."

Did I Test My Understanding with AI?

Yes, I used AI prompts to:

- Break down each part of the algorithm
- Confirm how scoring works
- Validate my explanation

Possible Improvements

- Make scoring weights configurable (not hardcoded)
- Reduce repeated datetime.now() calls (optimize performance)
- Improve readability by separating scoring rules into smaller functions
- Add comments or documentation for clarity